

Linux Server Operations Lab

Phase 3 復習資料：PythonからExcelを読み取る

プロジェクト構成：Excel → Python → Ansible → Linux

Phase 3では、Phase 2で作成したサーバ台帳をPythonから読み取り、検証し、Ansibleへ渡せるデータ構造に変換するところまで実施した。

1. Phase 3の目的

- Excelを単なる表ではなく、自動化システムの入力データとして扱えるようにする。
- openpyxlを使って.xlsxファイルを読み取り、シート・セル・行データを取得する。
- 1台分のサーバ情報をPythonの辞書（dict）へ変換し、項目名で扱えるようにする。
- Hostname、IP、Environment、RoleなどをPython側でも検証し、誤入力をAnsibleへ流さない土台を作る。

2. 今回の役割分担

要素	今回の役割
Excel	サーバ台帳の原本。ServerID / Hostname / IP / OS / Role / Environmentを保持。
Python	Excelを読み取り、辞書化・入力チェック・整形を担当。
Ansible	Phase 3では未実行。次PhaseでPythonが作った情報をInventoryへ変換する先。
Linux	controller上でPythonとopenpyxlを実行し、Excelファイルを保管。

3. 実施内容と主なコマンド

3-1 Excelファイルをcontrollerへ配置

```
scp Linux_Server_Operations_Lab.xlsx controller@192.168.56.10:/home/controller/  
ls -l ~/Linux_Server_Operations_Lab.xlsx
```

Mac上のExcelファイルをSSH経由でcontrollerへコピーし、/home/controller/配下に存在することを確認した。

3-2 / 3-3 Python Excelライブラリの準備

```
python3 -c 'import openpyxl; print(openpyxl.__version__)'
```

最初は ModuleNotFoundError

が発生。pipも未導入だったため、Ubuntu側でpython3-pipを導入した。さらにUbuntu 24.04のPEP 668によりpip install --user openpyxlが拒否されたため、OS管理方式に合わせて apt で python3-openpyxl を導入した。最終的に openpyxl 3.1.2 を確認した。

3-4 Excelブックとシートを開く

```
import openpyxl  
wb = openpyxl.load_workbook('/home/controller/Linux_Server_Operations_Lab.xlsx')  
print(wb.sheetnames)
```

結果：['01_SERVER_MASTER']。PythonからExcelブックそのものを正常に開けることを確認した。

3-5 1セルを読む

```
ws = wb['01_SERVER_MASTER']  
print(ws['A2'].value)
```

結果：SV001。Excelのセル値をPythonの値として取得できた。

3-6 / 3-7 1行・3台分を読む

```
[ws.cell(row=2, column=c).value for c in range(1, 7)]  
[print([ws.cell(row=r, column=c).value for c in range(1, 7)]) for r in range(2, 5)]
```

tokyo / yokohama / omiya の3台分を、Excelの行データとして取得した。

4. Pythonのデータ構造へ変換

Excelの1行を単なるリストで扱うと、「何番目がIPか」を順番で覚える必要がある。そこで、見出し行と値をzipし、dictへ変換した。

```
headers = [ws.cell(row=1, column=c).value for c in range(1, 7)]
values = [ws.cell(row=2, column=c).value for c in range(1, 7)]
server = dict(zip(headers, values))

{'ServerID': 'SV001', 'Hostname': 'tokyo', 'IP_Address': '192.168.56.11',
 'OS': 'Ubuntu 24.04', 'Role': 'web', 'Environment': 'DEV'}
```

辞書化後は `server["Hostname"]` や `server["IP_Address"]`

のように、列番号ではなく項目名で取得できる。Phase 4のInventory自動生成でこの形を利用する。

5. 入力チェック

チェック	目的	今回の結果
Hostname / IP空欄	接続に必要な必須項目が空でないか確認	3台ともOK
Environment	DEV / STG / PROD 以外を弾く	3台ともOK
Role	web / app / db 以外を弾く	3台ともOK
IP形式	ipaddress標準ライブラリで妥当なIPか確認	3台ともOK

入力チェックの意味

Excel側のドロップダウンだけに頼らず、Python側でも検証することで、誤入力や壊れたデータをAnsibleへ渡す前に止められる。自動化では「入力を信用しすぎない」ことが重要。

6. トラブルシュート記録

openpyxlが見つからない

何が起きた：ModuleNotFoundError: No module named openpyxl

原因：openpyxl未導入

修正：pipまたはOSパッケージ管理を確認

pipがない

何が起きた：/usr/bin/python3: No module named pip

原因：python3-pip未導入

修正：aptでpython3-pipを導入

apt installで404

何が起きた：古いパッケージURLへアクセスし404

原因：aptパッケージ一覧が古い

修正：sudo apt update後に再実行

pip installが拒否

何が起きた：externally-managed-environment

原因：Ubuntu 24.04のPEP 668によるシステムPython保護

修正：sudo apt install python3-openpyxl -yを利用

ws = ... がcommand not found

何が起きた：Python文をLinuxシェルに直接入力

原因：実行場所の違い

修正：python3 -c またはPythonスクリプト内で実行

7. 最終結果

```
SV001 tokyo      192.168.56.11 web DEV OK
SV002 yokohama  192.168.56.12 web DEV OK
SV003 omiya     192.168.56.13 web DEV OK
```

Excelのサーバ台帳3行をPythonが読み取り、辞書化し、主要項目を検証して、全サーバOKと判定できた。これでPhase 3の目標「Excel → Python」が完成した。

8. 今回学んだこと

- Excelのセルはopenpyxlを介してPythonの値として取得できる。
- ワークブック (wb) → ワークシート (ws) → セルという階層でExcelを扱う。
- リストより辞書型に変換したほうが、HostnameやIPを名前で参照できて安全。
- Excel側の入力規則とPython側の検証を組み合わせると、自動化の入力品質が上がる。
- UbuntuのシステムPythonではPEP 668を意識し、OS管理のパッケージはaptを優先する。
- LinuxシェルとPython文法は実行場所が異なる。

9. 実務での用途

- Excelで管理しているサーバ台帳をコードへ手入力せず、自動化処理の入力元にできる。
- IPやEnvironmentの誤入力を事前検証し、誤操作のリスクを減らせる。
- 環境・RoleによってAnsibleの対象や処理を分岐させる基礎になる。
- 台帳更新だけでInventoryや点検対象へ反映する設計へ発展できる。

10. note記事候補

- 「Excelのサーバ台帳をPythonで読む - openpyxl入門」
- 「Ubuntu 24.04でpip installがexternally-managed-environmentになった話」
- 「Excel→Python→Ansibleの前処理で入力チェックを入れる理由」
- 「Excelの1行をPythonのdictへ変換すると何が便利なのか」

次回：Phase 4

ExcelのHostname / IP_AddressなどをPythonで読み取り、Ansible Inventoryを自動生成する。ここで Excel → Python → Ansible の3層が初めて実際につながる。